

SIMATS ENGINEERING

CO1-ASSESSMENT TOOL 3 – CASE STUDY

Student Name: Bemanaveni Manish

Register Number: 192411149

Course Name: Internet Programming

Course Code: CSA4308

Scenario Overview: Web Development Standards Implementation

A startup building a modern web application encounters five distinct technical issues during development: embedding a promotional video, securely submitting registration data, structuring the navigation menu semantically, correcting XML syntax errors in configuration files, and selecting the correct protocol for web communication. Each issue is analysed below with reference to the code snippets provided, the correct standard is identified, and the reasoning behind the choice is justified.

Question 1: Correct HTML Tag for Video Embedding

Snippet 1 – Video Embedding Options

- A. `<media src="video.mp4"></media>`
- B. `<video src="video.mp4" controls></video>`
- C. `<movie src="video.mp4"></movie>`
- D. `<vid src="video.mp4"></vid>`

Answer

Option B – the `<video>` element with the controls attribute – is the correct choice.

Justification

The `<video>` tag is the native HTML5 element specifically designed for embedding video content in a browser without relying on external plug-ins such as Flash. Because it is a standards-compliant, W3C-recognised element, every modern browser understands and renders it consistently.

The remaining options use non-standard, invalid tags:

- `<media>`, `<movie>`, and `<vid>` are not part of the HTML5 specification, so browsers do not recognise them and will not render any video player.

- Only <video> exposes essential attributes such as controls (to show play/pause/volume controls), autoplay, loop, and multiple <source> children for format fallback, which are required for a functional, accessible media player on the homepage.

Recommendation: Use <video src="video.mp4" controls></video>, and ideally nest one or more <source> elements inside it to provide multiple formats (e.g., MP4 and WebM) for cross-browser compatibility.

Question 2: Appropriate HTTP Method for Form Submission

Snippet 2 – Form Submission Methods

- A. GET
- B. POST
- C. PUT
- D. DELETE

Answer

Option B – POST – is the appropriate HTTP method for submitting the registration form.

Justification

POST sends the form data inside the body of the HTTP request rather than appending it to the URL, which keeps sensitive information such as passwords, email addresses, and personal details out of browser history, server logs, and bookmarks.

Comparing the alternatives clarifies why POST is preferred for this scenario:

- GET appends data as visible query-string parameters in the URL, is length-restricted, and is meant for idempotent, cacheable retrieval of data — unsuitable and insecure for submitting confidential registration details.
- PUT is designed to create or completely replace an existing resource at a known URI (commonly used in RESTful update operations), not for the initial submission of a new registration form.
- DELETE is intended to remove a resource from the server and has no role in submitting new user data.

Recommendation: The registration form's <form> tag should specify method="POST" (and be served over HTTPS) so that user data is transmitted securely within the request body.

Question 3: Proper Semantic Element for Navigation

Snippet 3 – Navigation Elements

- A. <div>
- B. <nav>

C. <section>

D. <header>

Answer

Option B – the <nav> element – is the correct semantic tag for the navigation menu.

Justification

HTML5 introduced <nav> specifically to wrap the primary blocks of navigation links on a page, giving both browsers and assistive technologies a clear, machine-readable signal about the purpose of that content.

The other options fail to convey the correct meaning:

- <div> is a generic, non-semantic container; it carries no inherent meaning, so screen readers and search engines cannot infer that it holds navigation links.
- <section> represents a standalone thematic grouping of content (e.g., a chapter or tab panel), not a set of navigation links.
- <header> represents introductory content or a group of introductory/navigational aids for its nearest sectioning ancestor, but it is not itself the semantic marker for a list of navigation links — a <nav> is typically placed inside a <header>, not used in place of it.

Recommendation: Wrap the menu links in <nav> (optionally inside a <header>) to improve accessibility (screen-reader landmark navigation) and search-engine optimisation.

Question 4: Correct XML Syntax and Errors in Other Options

Snippet 4 – XML Syntax

A. <note><to>User</to><from>Admin</from>

B. <note><to>User</to><from>Admin</from></note>

C. <note><to>User</from></note>

D. <note to="User"></note>

Answer

Option B is the only well-formed XML document: <note><to>User</to><from>Admin</from></note>

Justification

A well-formed XML document must satisfy three core rules: it must have a single root element, every opening tag must have a matching, correctly nested closing tag, and every tag must be properly closed with both an opening "<"/>" and a closing "</ ... >" pair. Option B satisfies all three: the <note> root element opens and closes, and the <to> and <from> child elements are each properly opened and closed without overlapping.

The errors in the remaining options are:

- Option A: The root element <note> is opened but never closed. An XML parser will throw an "unexpected end of file" or "mismatched tag" error because every element must have a corresponding closing tag.
- Option C: The tags are improperly nested/mismatched — <to> is opened but closed with </from> instead of </to>. XML requires closing tags to match their corresponding opening tags exactly and to close in the reverse order they were opened.
- Option D: The closing tag for <note> is incomplete — it reads </note without the closing angle bracket ">", making the tag syntactically invalid and unparsable.

Recommendation: Adopt Option B as the template for the configuration files, and validate all XML files against a schema (XSD/DTD) or a linter during the build process to catch unclosed or mismatched tags before deployment.

Question 5: Correct Protocol for Web Communication

Snippet 5 – Web Communication Protocols

- A. FTP
- B. HTTP
- C. SMTP
- D. SNMP

Answer

Option B – HTTP (Hypertext Transfer Protocol, ideally its secure variant HTTPS) – is the correct protocol for the application's web communication.

Justification

HTTP is the foundational application-layer protocol that defines how web browsers and web servers exchange requests and responses (GET, POST, PUT, DELETE, etc.) for retrieving and submitting HTML pages, form data, and other web resources.

The other protocols serve entirely different purposes and are not suited to serving a web application:

- FTP (File Transfer Protocol) is designed for transferring files between a client and server, not for rendering interactive web pages or handling request/response cycles for a browser.
- SMTP (Simple Mail Transfer Protocol) is used exclusively for sending and relaying email messages between mail servers.
- SNMP (Simple Network Management Protocol) is used for monitoring and managing devices on a network (routers, switches, servers), not for delivering web content.

Recommendation: The application should communicate using HTTPS (HTTP layered over TLS/SSL) so that all requests — particularly the registration form's POST submission — are encrypted in transit, in addition to using the correct base protocol, HTTP.

Summary of Findings

- Video embedding – <video controls> – the native, standards-compliant HTML5 media element.
- Form submission – POST – sends data in the request body, keeping it secure and off the URL.
- Navigation menu – <nav> – the semantic landmark for a group of navigation links.
- XML configuration – Option B – the only option that is well-formed XML (fully closed, properly nested tags).
- Web communication – HTTP / HTTPS – the application-layer protocol built for browser–server web traffic.